

1. Introduction to Python Programming

- Python is an [interpreted](#), [object-oriented programming](#) language.
- It was created by Guido van Rossum in 1989, and released in 1991.
- ABC programming language is said to be the predecessor of Python language.
- Python is influenced by following programming languages:

*ABC language

*Modula-3

-This programming language was named as Python because. One comedy Series named- Monty Python's Flying Circus

History of Python

Python is a widely-used general-purpose, high-level programming language. It was initially designed by Guido van Rossum in 1991 and developed by Python Software Foundation.

It was mainly developed for emphasis on code readability, and its syntax allows programmers to express concepts in fewer lines of code.

In the late 1980s, history was about to be written. It was that time when working on Python started.

Soon after that, Guido Van Rossum began doing its application-based work in December of 1989 at Centrum Wiskunde&Informatica (CWI) which is situated in the **Netherlands**.

It was started firstly as a hobby project because he was looking for an interesting project to keep him occupied during Christmas.

The programming language in which Python is said to have succeeded is ABC Programming Language, which had interfacing with the Amoeba Operating System and had the feature of exception handling.

He had already helped to create ABC earlier in his career and he had seen some issues with ABC but liked most of the features.

After that what he did was really very clever. He had taken the syntax of ABC, and some of its good features.

It came with a lot of complaints too, so he fixed those issues completely and had created a good scripting language that had removed all the flaws.

The inspiration for the name came from BBC's TV Show – 'Monty Python's Flying Circus', as he was a big fan of the TV show and also he wanted a short, unique and slightly mysterious name for his invention and hence he named it Python!

The language was finally released in 1991. When it was released, it used a lot fewer codes to express the concepts, when we compare it with Java, C++ & C. Its design philosophy was quite good too. Its main objective is to provide code readability and advanced developer productivity. When it was released it had more than enough capability to provide classes with inheritance, several core data types exception handling and functions.

He was the "Benevolent dictator for life" (BDFL) until he stepped down from the position as the leader on 12th July 2018. For quite some time he used to work for Google, but currently, he is working at Dropbox.

Applications of Python:

Python is known for its general-purpose nature that makes it applicable in almost every domain of software development.

Here, we are specifying application areas where Python can be applied.

1) Web Applications:

We can use Python to develop web applications. It provides libraries to handle internet protocols such as HTML, XML, Email processing ,etc.

One of Python web-framework named Django is used on **Instagram**.

2) Desktop GUI Applications:

The GUI stands for the Graphical User Interface, which provides a smooth interaction to any application. Python provides a **Tk GUI library** to develop a user interface.

3) Console-based Application:

Console-based applications run from the command-line or shell. These applications are computer program which are used commands to execute.

4) Image Processing Application:

Python contains many libraries that are used to work with the image. The image can be manipulated according to our requirements.

5) Software Development:

Python is useful for the software development process. It works as a support language and can be used to build control and management, testing, etc.

6) Scientific and Numeric:

This is the era of Artificial intelligence where the machine can perform the task the same as the human. Python language is the most suitable language for Artificial intelligence or machine learning. It consists of many scientific and mathematical libraries, which makes easy to solve complex calculations.

*Python Features/ Characteristics :

1) Easy to Learn and Use

Python is easy to learn and use. It is developer-friendly and high level programming language.

2) Expressive Language

Python language is more expressive means that it is more understandable and readable.

3) Interpreted Language

Python is an interpreted language i.e. interpreter executes the code line by line at a time. This makes debugging easy and thus suitable for beginners.

4) Cross-platform Language

Python can run equally on different platforms such as Windows, Linux, Unix and Macintosh etc. So, we can say that Python is a portable language.

5) Free and Open Source

Python language is freely available at [official web address](#). The source-code is also available. Therefore it is open source.

6) Object-Oriented Language

Python supports object oriented concepts.

7) Large Standard Library

Python has a large and broad library and provides rich set of module and

functions for rapid application development.

8) GUI Programming Support

Graphical user interfaces can be developed using Python.

9) Integrated

It can be easily integrated with languages like C, C++, JAVA etc.

10) Dynamically Typed

Many programming languages need to declare the type of the variable before runtime. With Python, the type of the variable can be decided during runtime.

This makes Python a dynamically typed language.

For example, if you have to assign an integer value 20 to a variable "x", you don't need to write `int x = 20`. You just have to write `x = 15`.

11) Extensible

A programming language is said to be extensible if it can be extended to other languages. Python code can also be written in other languages like C++, making it a highly extensible language.

Python Garbage Collection

Garbage Collection is a process of automatically freeing the memory of objects OR it is a process of automatic recycling the memory objects/destroying the memory objects which are no longer referenced by compiler/or any variable.

In python when garbage collection occurs all the objects with reference count zero are deallocated from the memory. This memory is thus freed and is now available inside the program heap.

The reference count increases in the following scenarios:

- on assigning a new name to the object
- on passing the object to a function as an argument
- on using the object inside a container like a dictionary or a tuple

The reference count decreases when:

- a function to which the object was passed returns

- the object is removed from its container
- execution flow goes out of the scope inside which the object was declared

A routine function called garbage collector is executed frequently to perform the memory cleanup.

Reference Counting in Python

In python, reference counting means how many references are currently present to the current object from other objects. As discussed previously the reference count increases after the operations that add more references to the current object. Similarly, the count decreases on the operations of dereferencing the current object. The memory is freed when the count becomes zero.

To understand the concept of reference count look at the example below.

The Python memory manager is of greedy nature, whenever we assign a variable to another variable, the interpreter does not allocate new memory space into the private heap of the python. Instead, it adds a new reference to the existing object in the heap.

See how the reference count is increased

```
x = 10 #reference count of object 10 is 1 because it is referred by x
y = x #reference count of object 10 is now 2 because the value of x is stored
into variable y, i.e. variables x,y,refers same object 10
if id(x) == id(y):
print("x and y refer to same object")
```

The output of the above code is x and y refer to same object.

See how the reference count is decreased

```
x = 10 #reference count of object 10 is 1 because it is referred by x
y = x #reference count of object 10 is now 2 because the value of x is stored
into variable y, i.e. variables x,y,refers same object 10
```

```
y = y + 1 # here the variable y has new value is allocated and it is 11, so now y  
refers object 11 and refererence count of 11 is now 1
```

But, the reference count of object 10 is decreased by 1 because, it is only referred by variable x

```
if id(x) != id(y):  
print("x and y do not refer to the same object")
```

The output of the above code is x and y do not refer to the same object.

Here the variable y is allocated a new block of memory when an operation of addition is performed on it.

What is memory management?

Memory management in simple terms means , the process of providing memory required for your program to store data and freeing up unused data in memory is called memory management.

Providing memory is called memory allocation. Freeing up memory is called memory de-allocation.

In Python , Memory manager is responsible for allocating and de-allocating memory.

How memory is allocated in Python?

Python memory has two parts stack memory and heap memory

Stack Memory –all the methods/method calls, references are store in stack memory.

Heap Memory – all the objects are stored in heap memory.

Consider following example,

```
a=10 #inside main() function  
b=20
```

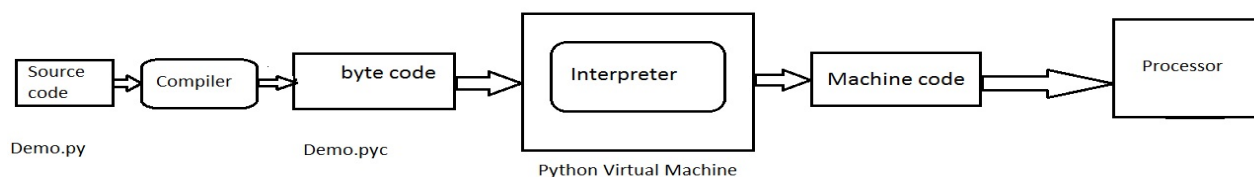
```
add()#calling add() function first time
add()#calling add() function second time
def add(): #def of add() function
    p,q=10,20
    print(p)
print(q)
```

Here, variable a,b are stored in stack memory, and the values/objects 10,20 are stored in heap memory

Then, the fun add() is called it also stored in stack memory(i.e. the variables p, q are stored in stack,) and the objects 10,20 are in heap. But the objects 10,20 are already in heap so, they are not stored into heap memory once again.

Then the fun add() is called once again but is not loaded into stack as well as heap memory because, all the references and its objects are already present into memory

*Python Virtual Machine(PVM):



- Computers understands only machine code i.e. 1s and 0s.Our source code is in high level language which needs to convert in machine level language.
- Compiler normally converts the source code into machinecode called as byte code size of each byte code (instruction) is 1 byte.
- Byte code is platform independent. The roll of Python Virtual Machine (PVM) is to convert byte code into machine code. Computer can execute this machine code and gives final output.
- The PVM is equipped with an interpreter.Interpreter checks and understand processor and OS.according to this interpreter converts byte code into machine code and sends that machine code to processor for execution.
- Since interpreter plays main role, often the PVM is also called as interpreter

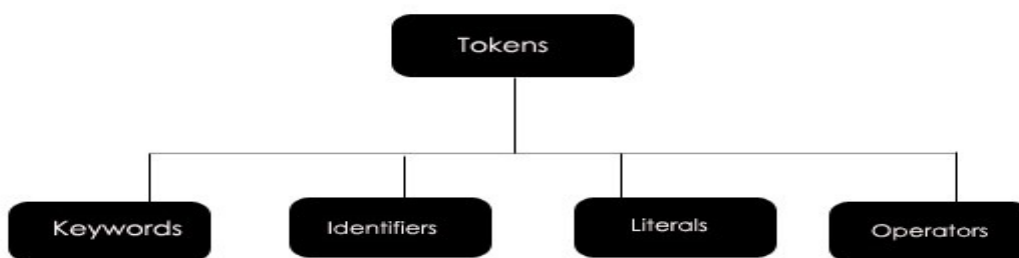
* Tokens In Python:

“Token is nothing but smallest individual unit of Python program.”

Python program has classes and every classes has some methods and methods contains executable statement and every executable statement contains the tokens i.e. statements are made up of several tokens.

Python supports 4 types of Tokens

- 1) Keywords
- 2) Identifiers
- 3) Literals
- 4) Operators



1) Keywords

The Keywords are reserved words, whose meaning is already known by Python compiler.

Each keyword have unique and fixed meaning and we are not able to change that meaning therefore they are also called as ‘Reserved Words’

Let’s see Python Keywords

<u>False</u>	<u>await</u>	<u>else</u>	<u>import</u>	<u>pass</u>
<u>None</u>	<u>break</u>	<u>except</u>	<u>in</u>	<u>raise</u>
<u>True</u>	<u>class</u>	<u>finally</u>	<u>is</u>	<u>return</u>
<u>and</u>	<u>continue</u>	<u>for</u>	<u>lambda</u>	<u>try</u>
<u>as</u>	<u>def</u>	<u>from</u>	<u>nonlocal</u>	<u>while</u>
<u>assert</u>	<u>del</u>	<u>global</u>	<u>not</u>	<u>with</u>
<u>async</u>	<u>elif</u>	<u>if</u>	<u>or</u>	<u>yield</u>

2) Python Identifiers

"Identifier is the name given by the programmer for any variable, package, class, interface, array, object etc."

It helps to differentiate one entity from another.

Rules for writing identifiers

i) Identifiers can be a combination of letters in lowercase or uppercase or digits or an underscore _.

ex. myClass, var_1 and print_this_to_screen, all are valid example.

ii) An identifier cannot start with a digit.

ex. 1name (It is invalid)

iii) Keywords cannot be used as identifiers.

ex. global = 1 (It is invalid because global is a keyword)

iv) We cannot use special symbols like !, @, #, \$, % etc. in our identifier.

v) An identifier can be of any length.

3) Literals

A literal represent a fixed value that is stored into variable directly in the program. They are represented directly in the code without any computation. Literals can be assigned to any variable.

e.gstr="welcome" or x=10 etc.

following are types of literals,

- String literals :- "hello" , '12345'
- Int literals :- 0,1,2,-1,-2
- Long literal- :: 89675L
- Float literals :- 3.14
- Complex literals :- 12j
- Boolean literals :- True or False
- Special literals :- None
- Unicode literals :- u"hello"
- List literals :- [], [5,6,7]
- Tuple literals :- (), (9,),(8,9,0)
- Dict literals :- {}, {'x':1}
- Set literals :- {8,9,10}

4) Operators

Operators are special symbols in Python that operates in operands. And the operands are nothing but Data values.

Python divides the operators in the following groups:

- i) Arithmetic operators
- ii) Assignment operators
- iii) Comparison operators
- iv) Logical operators
- v) Identity operators (Special Operator)
- vi) Membership operators (Special Operator)
- vii) Bitwise operators

i) Arithmetic operators

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, etc.

Operator	Meaning	Example
+	Add two operands or unary plus	$x + y + 2$
-	Subtract right operand from the left or unary minus	$x - y - 2$
*	Multiply two operands	$x * y$
/	Divide left operand by the right one (always results into float)	x / y
%	Modulus - remainder of the division of left operand by the right	$x \% y$ (remainder of x/y)
//	Floor division - division that results into whole number adjusted to the left in the number line	$x // y$
**	Exponent - left operand raised to the power of right	$x ** y$ (x to the power y)

Program for Arithmetic operators:

```
x=15
y=5
p=x+y
print("\n Addition=",p)
q=x-y
print("\n subtraction=",q)
```

ii) Assignment operators

Assignment operators are used in Python to assign values to variables.

The left side of assignment operator always contains a variable where right side can be value, variable or an expression.

Program for Assignments operators:

```
x=15
print("\n Assignment operator:",x)
x+=5 #same like x=x+5
print("\n Addition assignment:",x)
x/=5 #same like x=x/5
```

```
print("\n Division assignment:",x)
```

Operator	Example	Equivalent to
=	x = 5	x = 5
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5
//=	x //= 5	x = x // 5
**=	x **= 5	x = x ** 5

Table: Assignment operator

iii) Comparison operators

Comparison operators are used to compare values.

It returns either True or False according to the condition.

Operator	Meaning	Example
>	Greater than - True if left operand is greater than the right	x > y
<	Less than - True if left operand is less than the right	x < y
==	Equal to - True if both operands are equal	x == y
!=	Not equal to - True if operands are not equal	x != y
>=	Greater than or equal to - True if left operand is greater than or equal to the right	x >= y
<=	Less than or equal to - True if left operand is less than or equal to the right	x <= y

Program for comparison operator

```
x=15
y=20
if(x>y):
print("\n x is greater")
else:
print("\n Y is greater")
```

iv) Logical operators

Logical operators are used to compare two or more conditional statements.

Operator	Meaning	Example
and	True if both the operands are true	x and y
or	True if either of the operands is true	x or y
not	True if operand is false (complements the operand)	not x

-Here logical and operator returns true result if and only if both conditions are true.

-The logical or operator returns true result when both the conditions are true
Also returns true value when at least one condition is true

- The logical not operator complements the result .i.e. True converted in false
And false converted in true.

* Program for logicaland operator

```
x=15
```

```
y=20
if(x>y and x=15):
print("\n Condition is true")
else:
print("\n Condition is false")
```

Output: Condition is false

* Program for logical or operator

```
x=15
y=20
if(x>y or x=15):
print("\n Condition is true")
else:
print("\n Condition is false")
```

Output: Condition is True

*Program for logical and operator

```
x=15
y=20
ifnot(x>y):
print("\n Condition is true")
else:
print("\n Condition is false")
```

Output: Condition is True

v) Identity Operators

is and **is not** are the identity operators in Python. They are used to check if two values (or variables) are located on the same part of the memory. Two variables that are equal does not imply that they are identical.

Operator	Meaning	Example
is	True if the operands are identical (refer to the same object)	x is True
is not	True if the operands are not identical (do not refer to the same object)	x is not True

Program for is operator

```
x=20
y=20
if (x is y):
print("x and y have same memory location")
else:
print("x and y have diffrent memory location")
```

#output: x and y have same memory location

In above program variable x and y have same value i.e. 20, we know that if multiple variables have same values then a common memory location is allocated to those same valued variables.

is operator returns True when multiple variables have common memory location.

Program for is not operator

```
x=20
y=20
if (x is not y):
print("x and y have diffrent memory location")
else:
print("x and y have same memory location")
```

#output: x and y have samet memory location

is not operator returns True when x and y have **different** memory location.

vi) Membership operators

in and **not in** are the membership operators in Python. They are used to test whether a value or variable is found in a sequence

(string, list, tuple, set and dictionary).

In a dictionary we can only test for presence of key, not the value.

Operator	Meaning	Example
in	True if value/variable is found in the sequence	'H' in x
not in	True if value/variable is not found in the sequence	'H' not in x

Program for in operator

```
x = 'Hello world'
print('H' in x)
#Output: True
```

Program for in operator

```
x = 'Hello world'
print('z' in x)
# Output: True
```

vii) Bitwise operators

Bitwise operators act on operands as if they were strings of binary digits.

They operate bit by bit, hence the name is bitwise operators.

Following are bitwise operators

Note: Refer Truth table for Bit operation

In the table below: Let `x = 10` (`0000 1010` in binary) and `y = 4` (`0000 0100` in binary)

Operator	Meaning	Example
<code>&</code>	Bitwise AND	<code>x & y = 0</code> (<code>0000 0000</code>)
<code> </code>	Bitwise OR	<code>x y = 14</code> (<code>0000 1110</code>)
<code>~</code>	Bitwise NOT	<code>~x = -11</code> (<code>1111 0101</code>)
<code>^</code>	Bitwise XOR	<code>x ^ y = 14</code> (<code>0000 1110</code>)
<code>>></code>	Bitwise right shift	<code>x >> 2 = 2</code> (<code>0000 0010</code>)
<code><<</code>	Bitwise left shift	<code>x << 2 = 40</code> (<code>0010 1000</code>)

Program for Bitwise operators

```
x=10
y=4
print("\n output of bitwise and-",int(x&y));
print("\n output of bitwise or-",int(x|y));
print("\n output of bitwise not-",int(~x));
print("\n output of bitwise EXOR-",int(x^y));
print("\n output of bitwise shift right-",int(x>>y));
print("\n output of bitwise shift Left-",int(x<<y));
```

* Operator Precedence:

Precedence:-

It indicates the order in which the operators will be evaluated in an expression

The operator precedence in Python is listed in the following table. It is in descending order (upper group has higher precedence than the lower ones). Let's see operator precedence,

Operators	Meaning
()	Parentheses
**	Exponent
+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
==, !=, >, >=, <, <=, is, is not, in, not in	Comparisons, Identity, Membership operators
not	Logical NOT
and	Logical AND
or	Logical OR

* Associativity of Python Operators

Associativity:

It indicates /defines the order in which the operators of the same precedence will be evaluated in an expression

- We can see in the previous table that more than one operator exists in the same group. These operators have the same precedence.
- When two operators have the same precedence, associativity helps to determine the order of operations.
- Associativity is the order in which an expression is evaluated that has multiple operators of the same precedence. Almost all the operators have left-to-right associativity.
- For example, multiplication and floor division have the same precedence. Hence, if both of them are present in an expression, the left one is evaluated first.

* Left-right associativity in Python

In python almost all operator have left to right associativity except exponent(**) operator

```
# Output: 3
print(5 * 2 // 3)
# Shows left-right associativity
# Output: 0
print(5 * (2 // 3))
```

*Right-to-left associativity in Python

Exponent operator ** has right-to-left associativity in Python.

```
# Shows the right-left associativity of **
# Output: 512, Since  $2^{(3^2)} = 2^9$ 
print(2 ** 3 ** 2)
```

- # If 2 needs to be exponented first, need to use ()
Output: 64
print((2 ** 3) ** 2)

* Non associative operators

Some operators like assignment operators and comparison operators do not have associativity in Python. There are separate rules for sequences of this kind of operator and cannot be expressed as associativity.

Python Datatypes

Variable:“Variable is the name given to the memory location where the data is stored”.

Unlike other programming languages, Python has no command for declaring a variable. A variable is created the moment we first assign a value to it.

- There are several rules to declare the variable:
 - 1) Variable should not be keyword.
 - 2) Variable should not start with digit.
 - 3) Variable can be combination of alphabets, digits or underscore.
 - 4) Variable should not contain special symbol except underscore.
 - 5) Variable should not contain any white space character like horizontal tab, vertical tab, new line etc.
 - 6) Variable can be of any length.

Example

```
x = 5
y = "Sangola"
print(x)
print(y)
```

Here,

Variables do not need to be declared with any particular type and can even change type after they have been set.

String variables can be declared either by using single or double quotes:

-print() is a function which is used to print anything in python

There are two types of variables in python

1) Local Variable

2) Global variable

1) Local Variable:

The variables which are declared inside methods, constructors or blocks are called local variables.

These variables are declared and initialized within the method and they will be destroyed automatically when the method completes its execution.

2) Global variable

The variables which are declared outside methods, constructors or blocks are called global variables.

The scope of global variable is entire program.

We will discuss later in detail about local and global variables

Assigning multiple values to variables:

Python allows us to assign multiple values to multiple variables in one line.

```
x , y , z = "Python" , " Java" , "C++"
print(x)
print(y)
print(z)
```

Constants:

Constant is a value that remains the same and does not change or it cannot be modified.

Python does not allow us to create constants as easy as we do in C, C++ or Java. In Python, we can define constants in a module, where a module is a file which could be imported to access the defined constants and methods.

Note: A module constant should be in uppercase alphabets, where multi-word constant

Example:

Step 1 :Create a module and define a constant.

save module file by any name

Here we save module as **conex.py**

Defining constants in a module/file

INK = 'RED'

PI = 3.14

Step 2: import our module in python terminal or in new script and use it.

To access our constant following syntax will be used

Modulename.constatname

```
importconex
```

```
print('Value of constant APPLE :  
'conex.INK)
```

```
print('Value of constant GRAVITY :
```

Here first we imported our module as **importconex** after that we accessed our constants INK

and PI as **conex.INK** and **conex.PI**

Data Types In Python:

Data types are the classification or categorization of data items.

Python is a dynamically typed language so we need not to define data type of a variable while declaring it.

type() is a Built-In function used to know data type of any variable.

Since everything is an object in Python programming, data types are actually classes and variables are instance (object) of these classes.

Data Types Can be Built-In and User Defined .

Let's see Built-In Data types

There are various Built-In data types in Python.

Numeric Types: **int, float, complex**

Text Type: **string**

Sequence Types: **list, tuple**

Mapping Type: **Dict (Dictionary)**

Set Types: **Set**

Boolean Type: **Bool**

Binary Types: **bytes**

1)Numeric Type:

- There are three numeric types in Python:
int: for integral values
float: for floatl value
complex: for complex numbers
- Variables of numeric types are created when we assign a value to them.

```
x = 1 # int
y = 2.8 # float
z = 1j # complex

print(type(x))
print(type(y))
print(type(z))

#type() function is to know data type of any variable
```

Here we assigned numeric values to variables x,y,and z and we used **type()**function to know data types of variables x,y, and z.

Type Conversion:

Type Conversion is a process to convert one data type value in another.

We can convert from one type to another with the **int()**, **float()**, and **complex()** methods:

Example:

```
x = 1 # int
y = 2.8 # float

#convert from int to float:
a = float(x)

#convert from float to int:
b = int(y)

print(type(a))
print(type(b))
```

2) String:

String literals in python are surrounded by either single quotation marks, or double quotation marks or Triple quotation.

Ex. 'Hello' or "Hello" or """Hello"""

```
a = "Hello"
print(a)
```

- Like many other popular programming languages, strings in Python are arrays of bytes representing unicode characters.
- However, Python does not have a character data type, a single character is simply a string with a length of 1.
- We will see String in details in next chapter.

3) List:

A list is a collection of elements or items which is ordered and changeable.

List allows duplicate members.

In Python lists are written with square brackets. One list can contain another list as Element.

Example to create a List:

```
x = [1,2,3,"apple",[11,22]]
print(x)
```

- We will see more about List in next chapter.

4) Tuple:

- A tuple is a collection of elements which is ordered and **unchangeable**.
- Tuple allows duplicate members.
- In Python tuples are written with round brackets.
- A Tuple can contain another tuple.

Example to create a Tuple:

```
y = ("Sangola",1,55,3)
print(y)
```



5) Set:

- **Set** is a collection of elements which is unordered and indexed.
- There are no duplicate members in Set.
- In Python set is written with curly brackets
- Example to create a Set:

```
z = {5,10, "cherry"}  
print(z)
```

6) Dictionary:

- A dictionary is a collection which is unordered, changeable and indexed. Dictionary do not allows duplicate members.
- In Python dictionaries are written with curly brackets, and they have keys and values.
- Example

```
a = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 2019  
}  
  
print(a)
```

Here "brand","model" and "year" are the keys.

"ford","Mustang" and "2019" are the values

We will see more about Dictionary in next chapter.

7) Boolean Type:

- Booleans represent one of two values: True or False.
- In programming we often need to know if an expression is True or False.
- We can evaluate any expression in Python, and get one of two answers, True or False.
- When we compare two values, the expression is evaluated and Python returns the Boolean answer
- The bool() function allows us to evaluate any value, and give us True or False in return

```
print(10 > 9)
```

```
print(bool("Hello"))  
print(bool(0))
```



8) Bytes Type:

- The **bytes** type in Python is immutable and stores a sequence of values ranging from 0-255 (8-bits).

```
y=bytes(4)

print(y)

print(type(y))
```

*Control Statements/Structure:

The control structure is used to control flow of execution of python program.

The control structure are as follows

- If...else statements (Decision making)
- Looping (for loop and while loop)
- break and continue statements

a) Decision Making :

- Decision making is required when we want to execute a code only if a certain condition is satisfied.
- We can make decision with the help of,

if Statement

if...else statement

if...elif...else Statement

Nested if statements

Let's see them in brief

i) if statement:

- Python gives us a conditional ***if statement***, which is used when we want to test a *condition*.
- This *condition* is evaluated for a boolean value **true** or **false**.

Syntax

```
if(test_condition):
    statement(s)
```

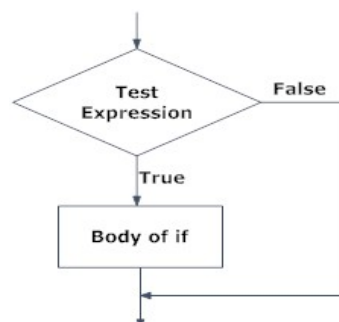


Fig: Operation of if statement

Example for if:

```
x=10
y=5
if(x>y):
    print("x is greater")
```

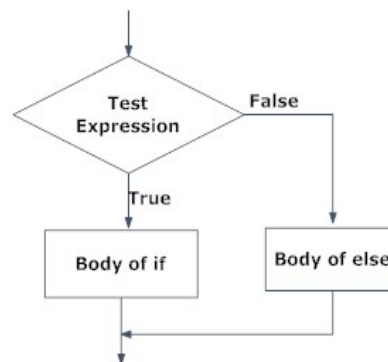
Output: X is greater

ii) if..else statement:

- The if..else statement evaluates test condition and will execute the body of if only when the test condition is True.
- If the condition is False, the body of else is executed.
- Indentation is used to separate the blocks.

Syntax of if...else

```
if(test_condition):
    Body of if
else:
    Body of else
```



Flowchart illustrating the execution of an if..else statement

Example for if..else

```
x=10
y=5
if(x>y):
    print("x is greater")
else:
    print("y is greater")
```

Output: X is greater

iii) if..elif..else statement

- The **elif** is short for else if. It allows us to check for multiple expressions.
- If the condition for if is False, it checks the condition of the next elif block and so on.
- If all the conditions are False, the body of else is executed.

Syntax of if...elif...else

```
if(test_condition):
    Body of if
elif(test_condition):
    Body of elif
else:
    Body of else
```

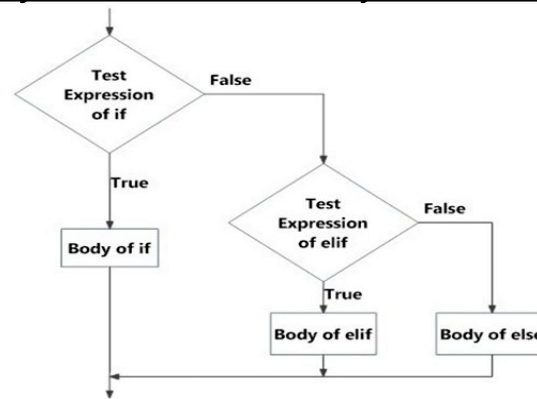


Fig: Operation of if...elif...else statement

Example for if..elif..else

```
per=55
if(per>70):
    print("Distinction")

elif(per>60 and per<=70):
    print("First class")

elif(per>50 and per<=60):
    print("Second class")

elif(per>40 and per<=50):
    print("Pass class")

else:
    print("fail")
```

Output: Second class

iv) Nested if statements:

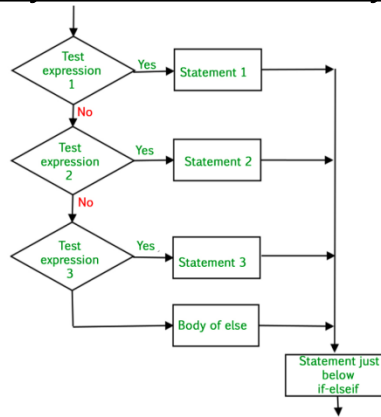
An if statement inside if statement is called as Nested if statement.

Any number of these statements can be nested inside one another.

Indentation is the only way to figure out the level of nesting. They can get confusing, so they must be avoided unless necessary.

Syntax for nested if

```
if(test_condition):
    body
if(test_condition):
    body
```



Example for nested if

```
x=120
if(x<200):
    print("value is less than 200")
    if(x<150):
        print("\n also less than 150")
```

Output-value is less than 200

also less than 150

b) Looping(Loops):

A Loop is sequence of instruction/s that repeated until certain condition is reached.

There are following loops present in python

i)for loop

ii) while loop

i) for loop:

- The for loop in Python is used to iterate over a sequence ([list](#), [tuple](#), [string](#)) or other iterable objects.
- Iterating over a sequence is called traversal.

Syntax of for Loop

for variable name in sequence:
Body or logic

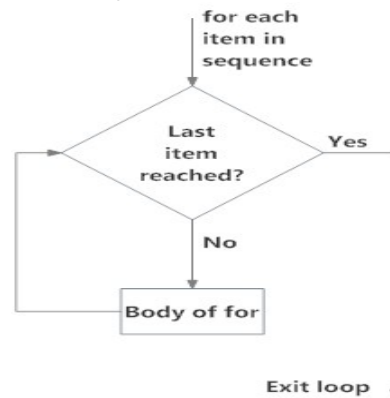


Fig: operation of for loop

Here each item from sequence is assigned to variable one by one. The loop terminates when there

Is no any item left in sequence.

Program of for loop

```
x=[0,1,5]
for p in x:
    print(p)
```

In our program x is a List(sequence) which have threeElements.

By using **for** loop each element of list is assigned to variable **p** after that these elements printed by function **print()**

for loop with else :

- A for loop can have an optional else block as well. The else part is executed if the items in the sequence used in for loop exhausts.
- The break keyword can be used to stop a for loop. In such cases, the else part is ignored.
- Hence, a for loop's else part runs if no break occurs.

Example

```
x=[0,1,5]
for p in x:
    print(p)
    #break
else:
    print("No items left")
```

for loop range() function:

We can generate a sequence of numbers using range() function. range(10) will generate numbers from 0 to 9 (10 numbers).

We can also define the start, stop and step size as range(start, stop-1,step_size-1). step_size defaults to 1 if not provided

```
x=[0,1,5]
for p in range(1,10):
    print(p)
```

We used range() function to create a sequence of numbers as, Range(1,10) where 1 is start and 10 is stop. By this a sequence if 1 to (10-1)=9is created. The step size is optional part

ii) **while loop** :The while loop in Python is used to iterate over a block of code as long as the test expression (condition) is true.

- We generally use this loop when we don't know the number of times to iterate beforehand.

Syntax of while Loop in Python

```
while (test_expression):
    Body of while
```

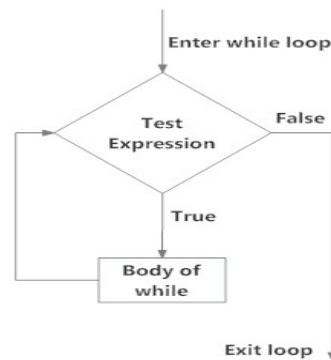


Fig: operation of while loop

Program to print 1 to 10 using while loop

```
i=1
while(i<=10):
    print(i)
    i+=1
```

While loop with else:

- Same as with [for loops](#), while loops can also have an optional else block.
- The else part is executed if the condition in the while loop evaluates to False.
- The while loop can be terminated with a [break statement](#).
- In such cases, the else part is ignored.
- Hence, a while loop's else part runs if no break occurs and the condition is false.

```
i=1
while(i<=10):
    print(i)
    i+=1
    break
else:
    print("else executed")
```

Output: 1

Our while loop iterates only one time
Because we used break statement.

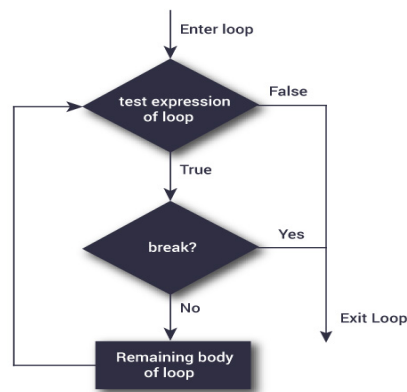
Also our else part don't get executed
Because of break statement

c) break and continue:

- In Python, break and continue statements can alter the flow of a normal loop.
- Loops iterate over a block of code until the test expression is false, but sometimes we wish to terminate the current iteration or even the whole loop without checking test expression.
- The break and continue statements are used in these cases.

* break:

- The break statement terminates the loop containing it.
- Control of the program flows to the statement immediately after the body of the loop.
- If the break statement is inside a nested loop (loop inside another loop), the break statement will terminate the innermost loop.



Program for break

```
for x in range(1,5):
    print(x)
    break
```

Output- 1

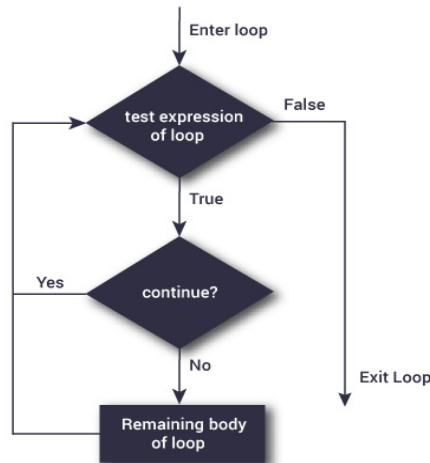
In our program for loop iterates only once
Because of break statement hence output is 1

* continue :

- The continue statement is used to skip the rest of the code inside a loop for the current iteration only.
- Loop does not terminate but continues on with the next iteration.

Program for continue

```
for val in "string":
    if val == "i":
        continue
    print(val)
    print("The end")
```



Python pass statement :

- In Python programming, the pass statement is a null statement.
- The difference between a [comment](#) and a pass statement in Python is that while the interpreter ignores a comment entirely, pass is not ignored.

However, nothing happens when the pass is executed. It results in no operation (NOP).

```
s = {'p', 'a', 's', 's'}
for val in s:
    pass
```

Assert keyword :

- The assert keyword is used when debugging code.
 - The assert keyword lets we test if a condition in our code returns True, if not, the program will raise an AssertionError and stop the execution of next block
- Syntax- assert condition, message
E.g. assert b!=0, "b has value is zero"

```
a=10
b=0
assert b!=0
print (a/b)
```

* Input and Output statements :

- In every programming language there is a need to get input from user and provide the output to user
- Here In python we can get input from user and provide the output for that some functions which are as follows

Python Output Using print() function

Python Input using input() function

- These two are built-in functions

print() function :

- We use the print() function to output data to the standard output device (screen).
- We can also output data to a file, but this will be discussed later.
- An example of its use is given below.

```
print('This sentence is output to the screen')
```

- Another example is given below:

```
a = 5  
print('The value of a is', a)
```

To display multiple values by using single print function

```
a=10
```

```
b=20
```

```
print(a,b)
```

OR

```
print("the value of a=",a,"the value of b=",b)
```

OR

```
print("the value of a={} and value of b={}".format(a,b))
```

Note- .format() function is applied on string which replace the values in string by specified values at the place of place holder (i.e. {})

input() function :

- Up until now, our programs were static. The value of variables was defined or hard coded into the source code.
- To allow flexibility, we might want to take the input from the user. In Python, we have the input() function to allow this.
- The syntax for input() is:

```
input([prompt])
```

- where prompt is the string we wish to display on the screen. It is optional.

```
s=input('enter any string')
```

Note: input() function accepts all inputs in string format so we have to do typecasting as per necessary.

e.g.

```
s=input('enter any integer value')
```

```
a=int(s)#explicit type casting
```

OR

```
s=int(input('enter any integer values'))
```

To get multiple inputs by using single input() function

```
a,b=input("enter any two values").split()
```

here, **.split() method** is applied on string, which is used to split/break given string

Syntax-

```
Input().split("separator",maxsplit)
```

Where,

separator- it indicates the separator of input is given(e.g. if separator is "," then input is given like 10,20) default separator comma(,) or space()

maxsplit- it indicates the max no of splits of given string(if maxsplit has value then it splits given string into 2 parts)

Note- both parameter of split() method are optional

* Nested Loop :

A loop inside loop is called as nested loop.

Program for nested loop

```
rows=6
for i in range(rows):
    for j in range(i):
        print('*',end=" ")
    print("\n")
```

- **type() function-**

It is used to display the type/ data type of particular variable

Syntax-

```
type(variable_name)
```

e.g. a=10

```
print("the type of variable a=",type(a))
```

- **id() function-**

It is used to display id/ reference/address of particular variable

Syntax-

```
id(variable_name)
```

e.g. a=10

```
print("the id of variable a=",id(a))
```

- **Type Casting-**

It is a process of converting one type/data type variable into another type/data type.

i.e. converting one caste of variable into another caste

Type casting has following two types..

1. Implicit casting 2. Explicit casting

1. Implicit casting-

In this type of casting the compiler converts one type to another type of variable without help of user i.e. automatically

e.g.

a=10 # here the type of a is int

a="hello" # here the compiler automatically converts the type of a int to string

2. Explicit casting-

In this type of casting the compiler converts one type to another type of variable with the help of user i.e. by using casting function

Syntax-

New_variable=casting_function_name(variable_name)

e.g.

a=input() # it reads string value i.e. type of a is string

b=int(a) # it converts string to int

Note- there are following different casting functions like,

int()- for casting in integer type

float()- for casting in float type

str()- for casting in string type
